PROJECT TECHNICAL REPORT: QUORIDOR AI MASTER

Course: Artificial Intelligence CSE472s

Spring 2026

Name	ID
Marina George Boushra	2300509
Karen Nasser Abd almlak	2300354
Zeina Wael Mahmoud	2301016
Habiba Mohamed Abdelmohsen	2300178
Habiba Ahmed Hashim	2300086
Shahinda Gamal Eldawi	2301029

Repository link: https://github.com/karennasser/AI_project

Demo link: https://drive.google.com/drive/folders/13PUsVTyVPefx4jyeKqxQ0QBXGcj0ZRgG?usp=drive_link

Backup Source Code: https://drive.google.com/drive/folders/11L4yg2O38EOj3BP3SZfM2OIGjTivV3aR?usp=sharing

The project source code is hosted on GitHub, while a backup copy is also provided on Google Drive to ensure accessibility in case of any technical issues.

1.Design Decisions and Architecture

To build a successful Quoridor game with a working computer opponent, the project needs a strong and organized blueprint. If we mixed the game rules, the visual screen drawings, and the artificial intelligence math all into one giant file, the code would become chaotic, difficult to understand, and full of bugs.

To avoid this problem, we applied the Model-View-Controller (MVC) framework. This framework splits the code into three completely separate layers:

1. **The Model (Data):** Storing the raw layout, piece positions, and wall data.
2. **The View (Visuals):** Drawing the grid blocks, pawns, buttons, and colors on your monitor.
3. **The Controller (Logic):** Managing the rules, validating jumps, tracking turns, and letting the AI think.

Core Game Data and State:

- **Board.h and Board.cpp**

  The Board class acts as the master memory bank for the game state. It is responsible for keeping track of the map size, where both players are standing, and the list of all permanent walls placed on the grid. When a player chooses an action, this class updates their coordinates and checks if specific grid squares are free. It also lets other parts of the program add temporary walls to test moves safely before making them final.

- **Wall.h and Wall.cpp**

  The Wall class represents a single blocking barrier that covers two adjacent grid edges. It is a simple data piece that holds exactly three values: the column position, the row position, and whether the wall lies flat horizontally or stands vertically. It provides quick check functions so the board can instantly verify which directions are blocked.

Rules and Path Calculations:

- **WallValidator.h and WallValidator.cpp**

  The WallValidator class serves as the referee for blocking pieces. Before allowing a player or the computer to drop a wall on the screen, this class runs three fast checks to make sure the wall fits inside the board, does not overlap or cross an existing barrier, and does not completely trap either player away from their finish line.

- **PathFinder.h and PathFinder.cpp**

  The PathFinder class is a path calculation tool that treats the board like a grid network. It uses an exploration method called Breadth-First Search to count the exact number of squares a pawn must traverse to reach its winning side. If a player is completely sealed in, it returns a value of negative one, which helps both the referee and the AI see if paths are open.

The Artificial Intelligence:

- **AIPlayer.h and AIPlayer.cpp**

  The AIPlayer class controls the brain of the computer opponent when you choose to play in solo mode. Depending on the difficulty level you select, it simulates future game turns inside a looking-ahead tree using the Minimax algorithm with Alpha-Beta pruning. It calculates and picks the exact move or wall placement that gives the computer the shortest remaining path while forcing the human player into the longest possible detour.

Graphics and User Interface:

- **gameboardwidget.h and gameboardwidget.cpp**

  The GameBoardWidget class is the visual display that draws the game elements on your screen. It translates raw mouse clicks into grid rows and columns, handles the code for drawing the squares, pawns, and walls, and colors valid squares green to show you where your piece can jump. It also records snapshots of the game data into history memory arrays so the Undo and Redo buttons work instantly.

- **mainwindow.ui, mainwindow.h, and mainwindow.cpp**

  The MainWindow class creates the empty display frame that opens up when you start the software. It sets up the core window structure and visual boundaries where the interactive parts of the game sit. The custom side panels and settings dropdowns are added to this window layout later on during the program initialization.

The Main Controller

- **GameController.h and GameController.cpp**

  The GameController class enforces the core rules of a live match. It manages turn switching, tracks how many wall resources each player has left, and monitors the target rows to see if someone crossed the finish line to win the match. It also contains the mathematical code required to validate complex pawn jumps, including straight overrides and diagonal dodges.

main.cpp

The main.cpp file is the main entry point that starts the entire program.

Key Design Choices and Why We Made Them

1. Fast Wall Detection

Instead of rebuilding a complex graph network every time a wall is placed, the program checks paths dynamically using a function called Board::isWallBetween. When a pawn wants to move, this function quickly checks the wall list to see if a wall is in the way. This is simple, uses very little memory, and prevents bugs.

2. Simple Undo and Redo Mechanics

To make the **Undo** and **Redo** buttons work smoothly, we created a lightweight struct called

```
struct GameState {
    std::vector<std::pair<int,int>> playersPos;
    std::vector<Wall> placedWalls;
    int p1Walls;
    int p2Walls;
};
```

We save these small data snapshots into storage arrays called QStack. Because we only save coordinates instead of heavy visual pictures, switching back and forth between turns is instant.

2. Implementation Challenges and Solutions

Challenge 1: Highlighting Diagonal and Jump Moves Dynamically

- **Problem:** In Quoridor, when two pawns are face-to-face, the rules change. A player can jump straight over the opponent if the space is open. But if a wall or the board edge blocks that space, the player must be allowed to jump diagonally instead.
- **Solution:** We wrote a step-by-step sorting tool inside GameBoardWidget::updateHighlights. It checks the spaces around the pawns and adds green highlights to the valid target tiles.

Challenge 2: Keeping the Visuals and Data Synchronized

- **Problem:** When a user changes settings on the sidebar (like choosing a smaller board size or shifting AI difficulty) in the middle of a live match, the system could crash or freeze up.
- **Solution:** We used clear connection lines (Qt Lambdas) inside main.cpp. When a user selects a new dropdown option, the program safely deletes the old board data, builds a fresh one instantly, and rewrites the screen.

3. AI Algorithm Explanation

The AI decides its moves using two main computer science tools: a pathfinder and a lookahead tree.

1. Shortest-Path Search (BFS)

The AI uses **Breadth-First Search (BFS)** to count exactly how many steps it needs to reach the winning side. Because every step on the board costs the same amount of effort (1 step), BFS is mathematically guaranteed to find the absolute shortest path. If a path hits a wall, that choice is skipped.

2. Point Scoring Heuristic

When the AI checks the board, it uses a simple point system to score the state:

Heuristic Score = Opponent's Path Length - AI's Path Length

- The AI wants a **high score**. It picks moves that make its own path shorter while making the human player's path longer.
- If a wall blocks someone completely (-1 path length), the AI triggers an immediate safety score (+99999 or -99999) to reject illegal moves.

3. Minimax Search with Alpha-Beta Pruning

The AI plays out future turns in its head recursively. Easy difficulty looks 1 turn ahead, Medium looks 2 turns, and hard looks 3 turns.

To avoid lagging, we added **Alpha-Beta Pruning**. If the AI finds a branch where the opponent can easily block it, it stops calculating that branch immediately (break). This keeps the AI thinking fast without slowing down the game.

4. Assumptions Made During Development

- **No Trapping Allowed:** We assume a wall placement is illegal if it completely seals off a player. Every player must always have at least one open path to their winning row.
- **Two-Player Setup:** The map is set up for a two-player vertical race. Player 1 starts at the top and wins at the bottom. Player 2 (the AI) starts at the bottom and wins at the top.
- **Strict Alternating Turns:** Players must take turns one after the other. The program switches turn back and forth using a direct toggle.
- **Standard Wall Sizes:** All walls are assumed to take up exactly two edge blocks on the grid map.

5. References and Resources Used

- Quoridor - How To Play
- Introduction to the A* Algorithm
- Tic Tac Toe: Understanding the Minimax Algorithm — Never Stop Building - Crafting Wood with Japanese Techniques